KenteCode AI Technologies

RAG Series Assignment: Project 1 — Build & Deploy a RAG Pipeline

KenteCode AI Academy

Property of KenteCode AI Technologies

Redistribution should retain this attribution

Assignment Overview

Kentecode AI Academy RAG Series Due: August 15, 2026

Section 0: Overview

In class you built **KenteCodeGPT** — a RAG assistant that answers questions about the KenteCode AI Academy website. In this project you will **replicate the entire pipeline end to end, on a document set of your own choosing**, and ship it as a real application: a FastAPI backend serving a REST API, with a Streamlit chat UI in front of it.

You are not starting from a blank page. You may reuse and adapt any function from lectures - the point of this project is proving you can put the pieces together yourself and reason about the choices you made, not inventing new algorithms.

Learning Objectives

By the end of this project you should be able to:

1. Parse, clean, and **chunk** real documents with a defensible strategy.
2. **Embed** chunks and index them in a vector database, with useful metadata attached.
3. Implement a **retrieval** pipeline (dense at minimum; hybrid + reranking for the advanced tier).
4. Generate **grounded answers with citations**, and refuse to answer when the context doesn't support it.
5. Expose the pipeline behind a **FastAPI** REST API with proper request/response models and error handling.

6. Build a usable **Streamlit** chat interface that talks to your API (not directly to the vector DB).
7. Manage secrets and configuration correctly (no API keys committed to git).

Section 1: Choose Your Dataset

Pick a document set you actually care about. Some ideas: a product's documentation site, a company handbook, a set of research papers,
your own portfolio of resumes/cover letters/projects, a game's rulebook etc.

Requirements:

- **Do not reuse the exact KenteCode AI website PDF from class.** Bringing your own corpus is the point.

Section 2: Part A — Indexing Pipeline

(mirrors part 2: Chunking, Embedding & Vector DB)

Build a script that, given your source documents, produces a populated vector index.

Requirements:

- Load and clean raw text from each source (strip headers/footers/boilerplate, fix whitespace, etc.).
- Chunk the text with a **documented** strategy — state your chunk size, overlap, and splitter choice
  (e.g. `RecursiveCharacterTextSplitter`, token-based splitting) and *why* you picked those numbers.
- Attach metadata to every chunk: at minimum `doc_id`, `title`/source name, and a chunk index. Include page numbers if your source format has them.
- Embed chunks with an embedding model of your choice (`text-embedding-3-small` is the class default; other providers are fine if you can justify them).
- Upsert into a vector database — Pinecone (used in class), or an alternative (Chroma, Qdrant, Weaviate, FAISS) if you prefer to self-host.
- Indexing must be **idempotent**: running it twice should not create duplicate chunks.
- Produce a short **indexing report** (a printed summary or a markdown cell is fine) covering: number of source docs, number of chunks, average chunk size, embedding dimension, and index name.

Section 3: Part B — Retrieval & Generation

(mirrors Part 3: Retrieval)

Core requirements (required for a passing grade):

- A `vector_search(query, top_k)` function that performs dense retrieval against your index.
- A `generate_answer(query)` function that retrieves context and produces an answer **grounded only in the retrieved chunks**,
  with inline citations back to specific chunks (e.g. `[1]`, `[2]`).
- The system must say it doesn't know when the retrieved context doesn't contain the answer — no answering from the model's general knowledge.
- Sparse/BM25 retrieval fused with dense results via Reciprocal Rank Fusion (hybrid search).
- Query rewriting and/or multi-query expansion before retrieval.
- Cross-encoder reranking as a second pass.
- A small RAGAS-style evaluation: build a golden set of **≥5 question/answer pairs** for your corpus and report faithfulness,
  answer relevancy, and context precision.

Section 4: Part C — FastAPI Backend

(mirrors part 5: Deployment)

Wrap your Part B pipeline in a FastAPI app.

Required endpoints:

Method	Path	Request	Response
`POST`	`/query`	`{ "question": str, "top_k": int? }`	`{ "answer": str, "sources": [{ "chunk_id", "title", "text", "score" }] }`
`GET`	`/health`	—	`{ "status": "ok" }`

Requirements:

- Use Pydantic models for request and response bodies (no loose dicts).
- Load configuration (API keys, index names, model names) from environment variables via `.env` — never hardcode secrets. Ship a `.env.example` with placeholder values.
- Handle errors gracefully: empty/missing question, upstream API failure with a clear error body, not an unhandled stack trace.
- Enable CORS so your Streamlit app (running on a different port/origin) can call it.

Section 5: Part D — Streamlit UI

Build a chat interface that talks to **your FastAPI backend** — the UI should never import your retrieval code or touch the vector DB directly.

Requirements:

- A chat-style interface (`st.chat_message` / `st.chat_input`) that posts to `/query` and renders the answer.
- Cited sources are visible — an expandable panel per message showing the chunk text, title, and score for each citation.
- Conversation history persists across turns via `st.session_state` .
- The backend API base URL is configurable (env var or a sidebar field), so a grader can point your UI at your deployed API without editing code.
- deploy both halves — e.g. the FastAPI backend on vercel and the Streamlit app on Streamlit Community Cloud.

Section 6: Deliverables

Submit a GitHub repository containing:

- Your working code
- README.md

Your `README.md` must describe:

- What the dataset is and where it came from.
- Your chunking strategy and why you chose it.
- How to reproduce indexing from scratch.

- How to run the API and the UI locally (exact commands).
- Which environment variables are required.
- Known limitations / things you'd improve with more time.

Also include a short demo — a **≤10 minutes screen recording ** describing your application and showing at least
3 real questions going in and a grounded, cited answer coming out.